

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Ecole Supérieure En Sciences Et Technologies De
L'informatique Et Du Numérique(ESTIN)



Rapport de Stage

BUDGESTIN - Digitalisation du Service Budgétaire de l'ESTIN

Encadré par :

Madame Samiha Ait Taleb

Dirigé par :

Madame Lylia Aberkane

Réalisé par :

Benlala Raid Athmane

Dennoub Aymen

Yerfa Belkacem

Année Universitaire : 2024 - 2025

Table des matières

Introduction	4
0.1 Contexte et problématique	5
0.1.1 Présentation de l'établissement d'accueil : l'ESTIN .	5
0.1.2 Présentation du service budgétaire	5
0.1.3 Problèmes rencontrés avec l'ancienne procédure (papier ou semi-numérisée)	5
0.1.4 Besoin de modernisation et d'automatisation	6
0.1.5 Importance de la numérisation pour gagner du temps et réduire les erreurs	6
0.2 Objectifs du projet	7
0.3 Présentation globale de la solution	9
0.4 Flux général de l'application	11
0.5 Technologies utilisées (vue d'ensemble)	12
0.6 Bénéfices attendus	13
0.7 Transition vers la partie technique	13
1 Documentation Technique FRONT	15
1.1 Aperçu du Projet	15
1.2 Architecture du Projet	15
1.2.1 Structure des Répertoires	15
1.3 Stratégie de Gestion d'État	16
1.3.1 État Local (Hooks React)	16
1.3.2 État Inter-Pages (Zustand)	16
1.3.3 État Serveur (@tanstack/react-query)	16
1.4 Composants Clés	17
1.4.1 Composant AsyncBuilder	17
1.5 Système de Contextes	18
1.5.1 Contexte de Thème	18
1.5.2 Contexte Utilisateur	18
1.6 Routage	19
1.7 Directives de Développement	19

1.7.1	Ajout de Nouvelles Fonctionnalités	19
1.7.2	Arbre de Décision pour la Gestion d'État	19
1.7.3	Bonnes Pratiques	20
1.8	Déploiement	20
1.9	Dépannage	20
1.9.1	Problèmes Courants	20
1.9.2	Outils de Développement	21
1.10	Hooks Personnalisés	21
1.10.1	useAxios	21
1.10.2	useGetRequest	21
1.11	Gestion des Erreurs	22
1.11.1	Utilitaires d'Erreurs	22
1.12	Fonctions Utilitaires	22
1.12.1	Utilitaires de Date	22
1.13	Optimisation des Performances	23
1.13.1	Optimisation des Requêtes	23
1.13.2	Gestion de la Mémoire	23
1.14	Hooks Personnalisés Additionnels	23
1.14.1	useSafeSideMode	23
1.14.2	useServerPagination	24
1.14.3	useIsMobile et useIsTablet	24
1.15	Implémentations des Stores	25
1.15.1	DataStore	25
1.16	Composants UI et Style	25
1.16.1	Système d'Animation	25
1.16.2	Composants shadcn/ui	26
1.17	Infrastructure	27
1.17.1	Configuration Nginx	27
1.17.2	Configuration Docker	27
1.17.3	Configuration de l'Environnement	28
1.18	Considérations de Sécurité	28
1.18.1	Flux d'Authentification	28
1.18.2	Autorisation	28
1.18.3	Sécurité des Données	28
1.19	Stratégie de Tests	28
1.19.1	Tests Unitaires	28
1.19.2	Tests d'Intégration	29
1.19.3	Tests E2E (de bout en bout)	29
1.20	Suivi et Analyse	29

1.20.1	Suivi des Erreurs	29
1.20.2	Surveillance des Performances	29
1.20.3	Analyses d'Utilisation	29
2	Backend – Implémentation et Architecture Technique	30
2.1	Aperçu du Projet et Technologies	30
2.2	Architecture Globale du Backend	30
2.2.1	Gestion de la Base de Données	33
2.2.2	Endpoints REST et Organisation des Routes	33
2.2.3	Authentification et Autorisation	34
2.2.4	Couche de Services (Logique Métier)	35
2.2.5	Couche de Repository (Accès aux Données)	36
2.2.6	DTO et Validation	37
2.2.7	Gestion des Erreurs et Journalisation	37
2.2.8	Sécurité et Bonnes Pratiques	38
2.2.9	Tests et Qualité du Code	38
2.2.10	Gestion de la Configuration	38
2.2.11	Problèmes Rencontrés et Solutions Apportées	39
2.2.12	Conclusion	40
	Conclusion Générale	41

Introduction

Ce rapport de stage présente le projet de digitalisation du service budgétaire de l'ESTIN, établissement de référence en informatique situé à Béjaïa. Dans un contexte où la gestion manuelle des demandes budgétaires engendrait des délais importants, un risque accru d'erreurs et un manque de transparence, il a été décidé de moderniser et d'automatiser ce processus. L'objectif principal était d'améliorer l'efficacité opérationnelle et la fiabilité des procédures en centralisant l'ensemble des informations dans une base de données et en facilitant la communication entre les différents services de l'établissement et le service budgétaire. Pour répondre à ce besoin, une application web a été conçue en adoptant une architecture moderne et modulaire, reposant sur un front-end développé en React, un back-end en Go, et une base de données PostgreSQL. L'application permet, d'une part, aux chefs de service (utilisateurs normaux) de soumettre et de suivre leurs demandes de besoins via une interface intuitive, et d'autre part, à l'administrateur (responsable des services budgétaires) de valider, commenter et gérer ces demandes à travers un tableau de bord complet. L'ensemble du projet a été orchestré à l'aide de Docker pour le déploiement et GitHub pour le suivi du code, garantissant ainsi une solution déployable et facilement maintenable. Ce rapport est structuré de manière à présenter d'abord le contexte et la problématique ayant motivé le projet, puis à détailler les objectifs et la solution globale mise en œuvre. Nous aborderons ensuite le flux général de l'application, les technologies utilisées et les bénéfices attendus. Enfin, une transition vers la partie technique détaillera la répartition des rôles et les contributions spécifiques de chaque membre de l'équipe : Benlala Raid Athmane pour la conception UI/UX et la coordination globale, Aymen Dennoube pour le développement front-end, et Belkacem Yerfa pour le développement back-end, avec une collaboration remarquable sur la configuration Docker. Ce projet de stage a ainsi permis d'apporter une réponse concrète et innovante aux défis rencontrés par le service budgétaire de l'ESTIN, tout en offrant une expérience pratique précieuse dans le domaine du développement web et de la gestion de projets

numériques.

0.1 Contexte et problématique

0.1.1 Présentation de l'établissement d'accueil : l'ESTIN

L'École Supérieure en Sciences et Technologies de l'Informatique et du Numérique (ESTIN) est une institution algérienne d'enseignement supérieur située à Amizour, à environ 25 km de Béjaïa. Elle est reconnue comme la troisième plus grande école d'informatique en Algérie. L'ESTIN forme des ingénieurs d'État en informatique, recrutant des bacheliers à l'échelle nationale. Après un tronc commun en informatique, les étudiants peuvent se spécialiser en intelligence artificielle, science des données ou cybersécurité.

0.1.2 Présentation du service budgétaire

Le service budgétaire de l'ESTIN est responsable de la planification, de l'allocation et du suivi des ressources financières de l'établissement. Ses principales missions incluent l'élaboration du budget annuel, le contrôle de l'exécution des dépenses et des recettes, ainsi que la garantie de la conformité aux réglementations financières en vigueur. Ce service joue un rôle essentiel dans la gestion efficace des ressources, assurant que les fonds sont utilisés conformément aux priorités stratégiques de l'école.

0.1.3 Problèmes rencontrés avec l'ancienne procédure (papier ou semi-numérisée)

Avant la mise en œuvre du projet de numérisation, le service budgétaire de l'ESTIN fonctionnait principalement avec des procédures papier ou semi-numérisées. Les principaux problèmes identifiés étaient :

- **Lenteur des processus** : Le traitement manuel des demandes entraînait des délais significatifs dans la validation et l'approbation des besoins.
- **Risque accru d'erreurs** : Les manipulations manuelles augmentaient la probabilité d'erreurs humaines, telles que des saisies incorrectes ou des pertes de documents.

- **Manque de traçabilité** : Il était difficile de suivre l'état d'avancement des demandes, ce qui pouvait entraîner des confusions et des retards supplémentaires.
- **Archivage inefficace** : La gestion physique des documents rendait l'archivage et la récupération d'informations fastidieux et peu fiables.

Ces défis sont couramment observés dans les organisations utilisant des processus budgétaires manuels. Par exemple, une étude a révélé que les entreprises font face à des difficultés telles que le manque de formation des responsables opérationnels, l'irréalisme des budgets et la longueur excessive des procédures budgétaires.

0.1.4 Besoin de modernisation et d'automatisation

Face à ces défis, il est apparu essentiel de moderniser et d'automatiser les procédures budgétaires de l'ESTIN. Les objectifs principaux de cette modernisation étaient :

- **Améliorer l'efficacité opérationnelle** : Réduire les délais de traitement et augmenter la productivité du personnel en automatisant les tâches répétitives.
- **Assurer une meilleure fiabilité** : Diminuer les erreurs humaines en standardisant les processus et en intégrant des contrôles automatisés.
- **Renforcer la transparence et la traçabilité** : Permettre un suivi en temps réel des demandes et des budgets, offrant ainsi une visibilité accrue aux parties prenantes.
- **Faciliter l'accès à l'information** : Centraliser les données pour un accès rapide et sécurisé, améliorant ainsi la prise de décision.

La numérisation des processus budgétaires est une tendance mondiale visant à améliorer la transparence, l'efficacité et la responsabilité des institutions publiques. Elle permet également de répondre aux attentes croissantes des citoyens en matière de services publics modernes et accessibles.

0.1.5 Importance de la numérisation pour gagner du temps et réduire les erreurs

La transition vers une solution numérique présente plusieurs avantages significatifs :

- **Gain de temps** : Les processus automatisés permettent de traiter les demandes plus rapidement, libérant ainsi du temps pour des tâches à plus forte valeur ajoutée.
- **Réduction des erreurs** : L'automatisation minimise les risques d'erreurs liés aux manipulations manuelles, garantissant une plus grande précision des données.
- **Amélioration de la collaboration** : Une plateforme numérique facilite la communication entre les différents services, assurant une meilleure coordination et une compréhension commune des processus.
- **Accès en temps réel** : Les parties prenantes peuvent consulter l'état des budgets et des demandes à tout moment, favorisant une prise de décision éclairée.

En somme, la numérisation du service budgétaire de l'ESTIN vise à moderniser les processus internes, améliorer l'efficacité opérationnelle et assurer une gestion financière transparente et fiable.

0.2 Objectifs du projet

Automatiser la soumission des demandes de besoins

Objectif : Remplacer le processus manuel de soumission des demandes de besoins par une solution numérique automatisée.

Détails :

- **Formulaires numériques** : Les utilisateurs pourront remplir des formulaires en ligne standardisés pour soumettre leurs demandes, réduisant ainsi les erreurs liées à la saisie manuelle.
- **Validation automatisée** : Des règles métier intégrées permettront de vérifier automatiquement la conformité des informations fournies, garantissant que seules les demandes complètes et correctes sont traitées.
- **Notifications en temps réel** : Les demandeurs recevront des notifications instantanées concernant l'état de leur demande, améliorant ainsi la communication et réduisant les délais d'attente.

Cette automatisation vise à améliorer l'efficacité opérationnelle en réduisant les délais de traitement et en minimisant les interventions manuelles.

Assurer une traçabilité et un suivi des demandes

Objectif : Mettre en place un système permettant de suivre chaque demande depuis sa soumission jusqu'à sa résolution finale.

Détails :

- **Historique des actions :** Chaque étape du processus sera enregistrée, offrant une visibilité complète sur les actions effectuées et les personnes impliquées.
- **Tableaux de bord personnalisés :** Les utilisateurs disposeront d'interfaces dédiées pour consulter l'état de leurs demandes, les échéances à venir et les actions requises.
- **Auditabilité :** La traçabilité complète garantit que toutes les actions sont documentées, facilitant les audits internes et externes.

Cette traçabilité renforcée permet d'identifier rapidement les goulots d'étranglement et d'assurer une transparence totale dans le traitement des demandes.

Centraliser les informations dans une base de données

Objectif : Regrouper toutes les données relatives aux demandes de besoins dans une base de données centralisée et sécurisée.

Détails :

- **Intégration des données :** Toutes les informations seront stockées dans une base de données PostgreSQL, assurant une cohérence et une intégrité des données.
- **Accès contrôlé :** Des niveaux d'accès différenciés garantiront que seules les personnes autorisées peuvent consulter ou modifier certaines informations sensibles.
- **Analyse et reporting :** La centralisation des données facilitera la génération de rapports détaillés et l'analyse des tendances, aidant à la prise de décisions stratégiques.

Cette centralisation vise à améliorer la gestion de l'information, à faciliter l'accès aux données et à soutenir une prise de décision éclairée.

En résumé, ces objectifs convergent vers une amélioration significative des processus budgétaires de l'ESTIN, en augmentant l'efficacité, la transparence et la fiabilité des opérations financières.

0.3 Présentation globale de la solution

Architecture globale

La solution repose sur une architecture moderne et modulaire, composée de trois couches principales :

- **Front-end** : Développé avec React, il offre une interface utilisateur réactive et ergonomique.
- **Back-end** : Construit en Go, il gère la logique métier et l'ensemble des traitements applicatifs.
- **Base de données** : La solution utilise PostgreSQL pour centraliser, sécuriser et assurer l'intégrité des données.

Cette architecture permet d'assurer une communication fluide entre les différents composants tout en garantissant une haute performance et une bonne scalabilité, sans entrer dans des détails techniques trop spécifiques.

Fonctionnalités principales

La solution web intègre plusieurs fonctionnalités clés qui couvrent l'ensemble du processus de gestion des demandes de besoins :

- **Création et soumission des demandes**
 - Un utilisateur normal peut soumettre une demande en précisant les caractéristiques du besoin : catégorie, nombre d'unités et prix estimé par unité.
 - La demande est initialement saisie via un formulaire numérique et peut être modifiée tant qu'elle n'a pas été confirmée.
- **Authentification et gestion des comptes**
 - Les utilisateurs normaux s'authentifient avec leur email, leur service et leur mot de passe.
 - La création d'un compte est suivie d'une validation par l'admin avant de pouvoir accéder pleinement à l'application.
 - Les utilisateurs ont la possibilité de modifier leur nom et leur mot de passe via leur espace personnel.
- **Processus de validation et suivi des demandes**
 - L'admin (le chef de service, responsable des services budgétaires) a la possibilité de consulter toutes les demandes soumises par les utilisateurs.

- Il peut valider, commenter ou même refuser une demande en fonction des critères budgétaires et administratifs.
- Un système de notifications informe les utilisateurs lors de la validation, de commenter ou du refus de leur demande, et notifie l'admin lors de la soumission d'une nouvelle demande ou de la création d'un compte.
- **Tableau de bord et reporting**
 - L'admin dispose d'un dashboard qui présente des indicateurs clés : le nombre total de demandes (validées ou non), ainsi que des graphiques illustrant les catégories de demandes les plus fréquentes par service.
 - Cette vue synthétique facilite la prise de décision et le suivi global des flux budgétaires.
- **Fonctionnalités complémentaires**
 - Possibilité d'imprimer une demande (.csv) pour un suivi ou une archivage physique.
 - Gestion des sessions et possibilité de déconnexion pour garantir la sécurité de l'application.

Utilisateurs ciblés et rôles

La solution est conçue pour répondre aux besoins des responsables de divers services de l'établissement. Les utilisateurs se répartissent en deux grandes catégories :

1. Utilisateurs normaux

Ces profils regroupent les chefs de service suivants :

- Directeur de l'école
- Directeur adjoint chargé des enseignants, des diplômés et de la formation continue
- Directeur adjoint chargé de la formation doctorale, de la recherche scientifique, du développement technologique, de l'innovation et de la promotion de l'entrepreneuriat
- Directeur adjoint chargé des systèmes d'information et de communication et des relations extérieures
- Directeur de la bibliothèque
- Chef de département préparatoire
- Chef de département spécialité

- Secrétaire générale
- Responsable de l'incubateur
- Médecin
- Responsable du centre de calcul
- Responsable de la sécurité
- Responsable du CDE
- Directeur de laboratoire

2. Admin

L'admin représente le responsable des services liés au budget, regroupant :

- Sous-directeur des finances et des moyens
- Service des marchés et des équipements
- Service des moyens, de l'inventaire et des archives

En résumé, la solution web vise à centraliser et à simplifier la gestion des demandes de besoins via une interface intuitive (React), une logique robuste (Go) et une base fiable (PostgreSQL), avec une distinction claire des rôles et responsabilités.

0.4 Flux général de l'application

Workflow simplifié (de la soumission à la validation finale)

1. Soumission de la demande

L'utilisateur (chef de service autorisé) accède à l'application via l'interface développée en React et complète un formulaire de demande de besoins. Les informations saisies incluent la description du besoin, la catégorie, le nombre d'unités et le prix estimé par unité.

2. Transmission et traitement

Une fois la demande soumise, celle-ci est envoyée au back-end (développé en Go) via une API sécurisée. Le back-end effectue des contrôles de validité et de cohérence des données avant de poursuivre le traitement.

3. Stockage dans la base de données

Les données validées sont ensuite stockées dans une base PostgreSQL. Ce stockage centralisé permet d'assurer l'intégrité et la pérennité des informations saisies.

4. Validation par l'admin

L'admin, représentant les responsables du service budgétaire, consulte l'ensemble des demandes soumises. Il peut alors valider, refuser, ou commenter chaque demande en fonction des critères budgétaires et des priorités de l'établissement.

5. Mise à jour et notifications

Une fois l'action réalisée par l'admin, une notification est automatiquement envoyée à l'utilisateur pour l'informer du statut de sa demande (validation, refus, demande de modification). L'utilisateur peut également modifier sa demande tant qu'elle n'est pas définitivement confirmée.

Interaction entre les différentes parties

- **Front-end (React)** : Gère l'interface utilisateur et permet la saisie des demandes. Il communique avec le back-end via des requêtes HTTP/REST.
- **Back-end (Go)** : Traite la logique métier (vérification, validation et mise à jour des demandes), gère l'authentification et orchestre les flux de données entre l'interface et la base de données.
- **Base de données (PostgreSQL)** : Assure la persistance des informations et permet la récupération rapide et fiable des données pour la consultation et l'analyse (tableau de bord, historique des actions, etc.).

Cette architecture intégrée garantit un flux de données fluide et sécurisé, tout en assurant une mise à jour en temps réel de l'état des demandes.

0.5 Technologies utilisées (vue d'ensemble)

- **Front-end** : React
- **Back-end** : Go

- **Base de données** : PostgreSQL
- **Hébergement** : Docker (avec le fichier Docker installé sur le serveur de l'école par l'ingénieur de l'ESTIN)
- **Gestion de code** : GitHub (pour le tracking et le versioning du code source)

0.6 Bénéfices attendus

- **Gain de temps** :
L'automatisation du processus de soumission et de validation permet de réduire les délais de traitement des demandes.
- **Réduction des erreurs humaines** :
La standardisation des formulaires et l'automatisation des vérifications limitent les risques d'erreurs lors de la saisie des informations.
- **Meilleure traçabilité et archivage des demandes** :
Le suivi détaillé de chaque demande (de la soumission à la validation) permet de constituer un historique complet, facilitant les audits et la consultation ultérieure.
- **Transparence entre les services** :
La centralisation des données et le système de notifications favorisent une communication claire et transparente entre l'ensemble des services concernés.

0.7 Transition vers la partie technique

Cette section marque la transition vers l'analyse détaillée des aspects techniques du projet. Elle présente brièvement la répartition des rôles et des responsabilités au sein de l'équipe, ainsi que quelques points forts sur la mise en place des outils techniques.

Répartition du travail entre les membres de l'équipe

- **Benlala Raid Athmane**
 - **Rôle** : UI/UX Designer
 - **Responsabilités** :
 - Conception de l'interface utilisateur
 - Conception de la base de données

- Extraction des besoins auprès des responsables de l'administration de l'ESTIN
- Organisation et participation aux réunions (meets)
- Gestion globale et coordination du projet
- **Aymen Dennoube**
 - **Rôle** : Développeur Front-end
 - **Responsabilités** :
 - Développement de l'interface utilisateur en React
 - Intégration des fonctionnalités côté client
 - Contribution à l'intégration de la solution globale
- **Belkacem Yerfa**
 - **Rôle** : Développeur Back-end
 - **Responsabilités** :
 - Développement de la logique métier en Go
 - Gestion des API et des interactions avec la base de données PostgreSQL

Collaboration technique et outils

Il est également à noter que Aymen et Belkacem ont collaboré pour créer et intégrer le fichier Docker. Cette démarche a permis de simplifier le déploiement et l'hébergement de l'application sur le serveur de l'école, assurant une configuration cohérente et moderne. Leur travail a été exemplaire, produisant un code propre et bien structuré qui contribue à la robustesse et à la maintenabilité de l'ensemble du système.

1 Documentation Technique FRONT

Ce document fournit une vue d'ensemble technique complète de l'application frontend BudgESTIN, en se concentrant sur son architecture, ses composants clés et ses pratiques de développement. Cette documentation est destinée aux développeurs assurant la maintenance ou l'extension du projet.

1.1 Aperçu du Projet

BudgESTIN est une application React moderne construite avec :

- **Vite** : Pour une construction et un développement rapides.
- **TailwindCSS** et **shadcn/ui** : Pour le style.
- **React Router v7** : Pour le routage.
- **@tanstack/react-query** : Pour la récupération et la mise en cache des données.
- **Zustand** : Pour la gestion d'état inter-pages.

1.2 Architecture du Projet

Le projet suit une architecture **axée sur les fonctionnalités** (feature-first), organisant le code par fonctionnalités métiers plutôt que par préoccupations techniques. Cette approche améliore la maintenabilité et rend le code plus intuitif à parcourir.

1.2.1 Structure des Répertoires

```
./src
modules/           # Feature modules
  admin/           # Admin feature
    dashboard/
    services/
```

```
    permissions/  
  auth/          # Authentication feature  
  user/          # User feature  
  shared/        # Shared features  
components/      # Shared UI components  
context/         # React contexts  
hooks/           # Custom hooks  
services/        # API services  
store/           # Zustand stores  
utils/           # Utility functions
```

Chaque module de fonctionnalité contient :

- Des composants spécifiques à la fonctionnalité.
- Des composants de page.
- Des hooks et utilitaires spécifiques à la fonctionnalité.
- Des types spécifiques à la fonctionnalité.

1.3 Stratégie de Gestion d'État

BudgESTIN emploie une approche hybride de gestion d'état :

1.3.1 État Local (Hooks React)

- Utilisé pour l'état spécifique aux composants.
- Gère l'état de l'interface utilisateur, les données de formulaire et les données temporaires.
- Exemple : Valeurs des champs de formulaires, états des modales, états de chargement.

1.3.2 État Inter-Pages (Zustand)

- Gère l'état persistant entre plusieurs pages.
- Utilisé pour l'état global spécifique aux fonctionnalités.
- Exemple : Préférences utilisateur, filtres sélectionnés.

1.3.3 État Serveur (@tanstack/react-query)

- Gère toute la récupération et la mise en cache des données serveur.

- Assure la synchronisation de l'état avec le serveur.
- Fournit des mises à jour automatiques en arrière-plan.
- Exemple : Données utilisateur, demandes de budget, notifications.

1.4 Composants Clés

1.4.1 Composant AsyncBuilder

Le composant **AsyncBuilder** est un utilitaire crucial pour gérer la récupération asynchrone de données et les états de rendu. Il fournit une manière cohérente de gérer les états de chargement, d'erreur et de succès dans toute l'application.

```
type AsyncBuilderProps<T> = {
  data: T | undefined;
  isLoading: boolean;
  isError: boolean;
  error: Error | null;
  children: (data: T) => React.ReactNode;
  loadingComponent?: React.ReactNode;
  errorComponent?: React.ReactNode;
};
```

// Usage example:

```
<AsyncBuilder
  data={userData}
  isLoading={isLoading}
  isError={isError}
  error={error}
  loadingComponent={<CustomLoading />}
  errorComponent={<CustomError />}
>
  {(data) => <UserProfile user={data} />}
</AsyncBuilder>
```

Fonctionnalités clés :

- Gestion des données typées (**type-safe**).
- États de chargement et d'erreur cohérents.
- Composants de chargement et d'erreur personnalisables.
- Modèle **render prop** propre pour l'état de succès.

1.5 Système de Contextes

1.5.1 Contexte de Thème

Gère l'état du thème de l'application et fournit des utilitaires liés au thème :

```
const ThemeContext = createContext<ThemeContextType>({  
  theme: 'light',  
  toggleTheme: () => {},  
});
```

// Usage:

```
const { theme, toggleTheme } = useTheme();
```

Fonctionnalités :

- Basculement mode clair/sombre.
- Persistance du thème à travers les sessions.
- Détection du thème système.

1.5.2 Contexte Utilisateur

Gère l'état de l'utilisateur authentifié et de la session :

```
const UserContext = createContext<UserContextType>({  
  user: null,  
  setUser: () => {},  
  logout: () => {},  
});
```

// Usage:

```
const { user, logout } = useUser();
```

Fonctionnalités :

- État d'authentification de l'utilisateur.
- Gestion du profil utilisateur.
- Gestion de session.

1.6 Routage

L'application utilise **React Router v7** pour la navigation, avec :

- Des routes imbriquées pour des mises en page complexes
- Des routes protégées pour l'authentification
- Un contrôle d'accès basé sur les rôles

Exemple de structure de routes :

```
<Routes>
  <Route element={<AuthRoute />}>
    <Route path="/" element={<LandingPage />} />
    <Route path="auth/*" element={<AuthPage />} />
  </Route>

  <Route element={<PrivateUserRoute />}>
    <Route path="user/*" element={<UserPage />} />
  </Route>

  <Route element={<PrivateAdminRoute />}>
    <Route path="admin/*" element={<AdminPage />} />
  </Route>
</Routes>
```

1.7 Directives de Développement

1.7.1 Ajout de Nouvelles Fonctionnalités

1. Créer un nouveau répertoire sous `src/modules/[nom-de-la-fonctionnalité]`
2. Ajouter les composants spécifiques à la fonctionnalité dans le répertoire de la fonctionnalité
3. Créer les hooks et utilitaires nécessaires
4. Mettre à jour le routage si nécessaire
5. Ajouter tous les stores Zustand requis

1.7.2 Arbre de Décision pour la Gestion d'État

Lors de l'ajout d'un nouvel état, suivre ces directives :

1. **L'état est-il spécifique à un seul composant ?**
Utiliser les hooks React (`useState`, `useReducer`)

2. L'état doit-il persister à travers les pages ?

Utiliser un **store Zustand**

3. L'état concerne-t-il des données serveur qui doivent être mises en cache ?

Utiliser **React Query**

1.7.3 Bonnes Pratiques

- Utiliser **TypeScript** pour tout nouveau code
- Suivre l'**architecture axée sur les fonctionnalités**
- Tirer parti d'**AsyncBuilder** pour la récupération de données
- Utiliser les **contextes** avec parcimonie, préférer **Zustand** pour l'état global
- Garder les **composants petits** et ciblés
- Écrire des **noms de composants et de fonctions significatifs**
- Documenter la **logique complexe** et les règles métiers

1.8 Déploiement

L'application est conteneurisée à l'aide de **Docker** et peut être déployée en utilisant :

Construire le conteneur

```
docker build -t budgestin-frontend .
```

Exécuter le conteneur

```
docker run -p 80:80 budgestin-frontend
```

La configuration **Nginx** est incluse dans le dépôt et gère :

- La diffusion de fichiers statiques
- Le routage SPA
- Le proxy API

1.9 Dépannage

1.9.1 Problèmes Courants

1. **Données Périmées** : Si les données semblent obsolètes, vérifier l'invalidation du cache de **React Query**.

2. **Problèmes d'Authentification** : Vérifier l'état du Contexte Utilisateur et la gestion des tokens.
3. **Problèmes de Routage** : S'assurer que les routes protégées sont correctement configurées.
4. **Gestion d'État** : Utiliser **React DevTools** pour inspecter l'état des composants et les contextes.

1.9.2 Outils de Développement

- **React DevTools** pour l'inspection des composants.
- **React Query DevTools** pour l'inspection du cache.
- **Browser DevTools** pour la surveillance du réseau et des performances.

1.10 Hooks Personnalisés

1.10.1 useAxios

Un hook central qui fournit une instance Axios configurée avec :

- La gestion automatique du rafraîchissement des tokens.
- Des intercepteurs d'erreurs avec des notifications toast.
- La configuration de l'URL de base.
- La journalisation des requêtes et réponses.

```
const axios = useAxios();  
await axios.get('/api/endpoint');
```

1.10.2 useGetRequest

Un wrapper autour de React Query pour les requêtes GET :

```
const { data, isLoading, error, refetch } =  
  ↪ useGetRequest<ResponseType>(  
    ['queryKey'],  
    '/api/endpoint',  
    config,  
    transformFn  
  );
```

Fonctionnalités :

- Réponses de type sécurisé.
- Mise en cache automatique.
- Gestion de l'état de chargement.
- Transformation de la réponse optionnelle.

1.11 Gestion des Erreurs

L'application utilise une approche centralisée pour la gestion des erreurs.

1.11.1 Utilitaires d'Erreurs

```
export const extractErrorMessage = (  
  error: unknown,  
  defaultMessage = "Une erreur s'est produite"  
): string => {  
  // Extrait des messages d'erreur conviviaux à partir de  
  ↪ divers types d'erreurs  
};  
  
export function handleError(error: any, toast: any, navigate:  
  ↪ any) {  
  // Gestion centralisée des erreurs avec navigation et  
  ↪ notifications  
}
```

Fonctionnalités clés :

- Extraction cohérente des messages d'erreur.
- Notifications toast pour le retour d'information à l'utilisateur.
- Navigation automatique pour les erreurs d'authentification.
- Gestion des erreurs de type sécurisé.

1.12 Fonctions Utilitaires

1.12.1 Utilitaires de Date

```
dateFormatted(date?: Date): string           // MM/dd/yyyy  
dateTimeFormatted(date?: Date): string       // MM/dd/yyyy HH:mm  
delay(ms: number): Promise<void>           // Utilitaire de  
  ↪ délai asynchrone
```

1.13 Optimisation des Performances

1.13.1 Optimisation des Requêtes

1. Stratégie de Mise en Cache

- React Query pour la mise en cache automatique.
- Délais d'obsolescence configurables.
- Mises à jour des données en arrière-plan.

2. Déduplication des Requêtes

- Déduplication automatique des requêtes via React Query.
- Cache partagé entre les composants.

3. Fractionnement du Code (Code Splitting)

- Fractionnement du code basé sur les routes.
- Composants chargés paresseusement.
- Imports dynamiques pour les fonctionnalités lourdes.

1.13.2 Gestion de la Mémoire

1. Nettoyage des Ressources

- Nettoyage approprié des `useEffect`.
- Gestion des abonnements.
- Nettoyage des timers.

2. Gestion d'État

- État local pour les composants UI.
- Zustand pour l'état global.
- React Query pour l'état serveur.

1.14 Hooks Personnalisés Additionnels

1.14.1 `useSafeSideMode`

Un hook spécialisé pour gérer les transitions de mode de panneau latéral en toute sécurité avec des boîtes de dialogue de confirmation :

```
const { safeSideMode, needsConfirmation } = useSafeSideMode();
```

Fonctionnalités

- Transitions sécurisées entre les modes de panneau
- Boîtes de dialogue de confirmation pour les modifications non enregistrées
- Intégration avec le store Zustand

1.14.2 useServerPagination

Un hook pour gérer l'état de pagination côté serveur :

```
const {
  currentPage,
  nextPage,
  previousPage,
  goToPage,
  totalPages,
  totalCount,
  setTotalCount
} = useServerPagination({
  initialPage: 1,
  perPage: 10
});
```

Fonctionnalités

- Contrôles de navigation entre les pages
- Gestion du nombre total d'éléments
- Gestion automatique des limites de pages

1.14.3 useIsMobile et useIsTablet

Des hooks de conception réactive pour détecter la taille de la fenêtre d'affichage de l'appareil :

```
const MOBILE_BREAKPOINT = 768;
const TABLET_BREAKPOINT = 1024;

// For mobile detection
const isMobile = useIsMobile();
```

```
// For tablet detection
const isTablet = useIsTablet();
```

Fonctionnalités

- Détection en temps réel de la taille de la fenêtre d’affichage
- Mises à jour automatiques de l’état lors du redimensionnement de la fenêtre
- Gestion cohérente des points de rupture (breakpoints)
- Démontage propre avec le nettoyage du listener d’événement

1.15 Implémentations des Stores

1.15.1 DataStore

Un store Zustand générique pour la gestion de collections de données :

```
const dataStore = useDataStore<YourType>();
```

Fonctionnalités

- Support des types génériques
- États de chargement et d’erreur
- Récupération d’éléments par ID
- Mises à jour automatiques de l’état

1.16 Composants UI et Style

1.16.1 Système d’Animation

L’application utilise une combinaison d’animations CSS et de composants React pour des transitions fluides.

Composant AnimatedMount

Un composant réutilisable pour des animations de montage/démontage fluides :

```
<AnimatedMount show={isVisible}>
  {children}
</AnimatedMount>
```

Fonctionnalités

- Transitions de hauteur fluides
- Animations d'opacité
- Temporisation et courbes d'accélération configurables
- Intégration de Framer Motion

Animations Globales

Animations personnalisées définies dans **animations.css** :

- Valeurs par défaut de transition globale avec durée personnalisable
- Keyframes d'animation communs (fondu-enchaîné, glissé-déposé)
- Fonctions d'accélération cohérentes
- Transformations accélérées par le matériel

1.16.2 Composants shadcn/ui

Le projet utilise shadcn/ui pour des éléments d'interface utilisateur cohérents :

- Variantes de boutons
- Contrôles de formulaire
- Composants de dialogue
- Notifications toast
- Menus déroulants

Personnalisation La personnalisation est gérée par :

- La configuration **components.json**
- L'extension du thème TailwindCSS
- Les variantes de composants personnalisées

1.17 Infrastructure

1.17.1 Configuration Nginx

La configuration Nginx de l'application gère :

```
[language=nginx] http include /etc/nginx/mime.types; default_type application/octet-stream;
server listen 4173;
location / { root /usr/share/nginx/html; index index.html index.htm;
try_files $uri $uri/ /index.html;
location /api/ { proxy_pass http://backend:5000/api/v1/; proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
```

Fonctionnalités clés

- Diffusion de fichiers statiques avec des types MIME appropriés
- Repli de routage SPA avec `try_files`
- Configuration du proxy API vers le service backend
- Forwarding de l'adresse IP réelle
- Optimisation des connexions worker

1.17.2 Configuration Docker

L'application utilise un processus de build multi-étapes défini dans le Dockerfile :

1. **Étape de build** (node :18-alpine)
 - Utilise `pnpm` pour une gestion de paquets plus rapide et plus efficace
 - Supporte les variables d'environnement au moment du build (ex : `VITE_API_URL`)
 - Optimisé pour un contexte de build plus petit
 - Produit un build prêt pour la production dans le répertoire `dist`
2. **Étape de production** (nginx :alpine)
 - Image Nginx légère basée sur Alpine
 - Copie les assets construits à partir de l'étape de build
 - Inclut une configuration Nginx personnalisée
 - Expose le port 4173 pour le trafic web
 - Exécute Nginx en mode foreground

1.17.3 Configuration de l'Environnement

L'application utilise un système de configuration hiérarchique :

1. Fichiers `.env` pour les variables d'environnement
2. `vite.config.ts` pour la configuration du build
3. Injection de configuration au runtime

Variables d'environnement clés

- `VITE_API_URL` : Point de terminaison de l'API backend
- `VITE_APP_ENV` : Nom de l'environnement
- `VITE_AUTH_DOMAIN` : Domaine d'authentification

1.18 Considérations de Sécurité

1.18.1 Flux d'Authentification

- Authentification basée sur les tokens
- Rotation des tokens de rafraîchissement
- Gestion sécurisée des cookies

1.18.2 Autorisation

- Contrôle d'accès basé sur les rôles
- Flags de fonctionnalité
- Routes protégées

1.18.3 Sécurité des Données

- Application HTTPS
- Prévention XSS
- Protection CSRF

1.19 Stratégie de Tests

1.19.1 Tests Unitaires

- Tests de composants avec React Testing Library

- Tests de hooks avec des rendus personnalisés
- Tests de stores avec des assertions d'état

1.19.2 Tests d'Intégration

- Tests d'intégration API
- Tests de navigation de routes
- Tests de gestion d'état

1.19.3 Tests E2E (de bout en bout)

- Tests des chemins critiques
- Validation des flux utilisateurs
- Compatibilité cross-navigateur

1.20 Suivi et Analyse

1.20.1 Suivi des Erreurs

- Implémentation de limites d'erreur (Error boundaries)
- Journalisation et rapport d'erreurs
- Collecte des retours utilisateurs

1.20.2 Surveillance des Performances

- Suivi des Core Web Vitals
- Surveillance de la taille du bundle
- Suivi du temps de réponse de l'API

1.20.3 Analyses d'Utilisation

- Suivi du comportement utilisateur
- Métriques d'utilisation des fonctionnalités
- Métriques de performance

2 Backend – Implémentation et Architecture Technique

2.1 Aperçu du Projet et Technologies

Le backend de BudgESTIN est développé en Go (Golang), un langage compilé et typé statiquement, offrant :

- Concurrence légère : Gestion de milliers de goroutines simultanément.
- Garbage Collection optimisée : Pauses minimales adaptées aux services en temps réel.
- Compilation rapide : Facilite le déploiement sur divers environnements.
- Écosystème riche : Bibliothèques standards puissantes et gestion des modules (Go Modules).

Principales bibliothèques et outils utilisés

- GORM : Simplifie les opérations CRUD avec la base de données.
- **golang-migrate** ou **goose** : Pour la gestion des migrations de schéma.
- **Validator** (ex. **go-playground/validator**) : Pour la validation des données entrantes.
- Framework HTTP (ex. Echo, Gin ou Chi) : Pour la gestion des routes REST.
- Logrus/Zap : Pour la journalisation.
- Viper/godotenv : Pour la gestion de la configuration via fichiers **.env** et variables d'environnement.

2.2 Architecture Globale du Backend

Le backend adopte une approche modulaire inspirée des concepts hexagonaux ou clean architecture. La structure de dossiers est organisée en trois niveaux principaux :

- **internal** : Logique métier et implémentation interne.
- **pkg** : Modules et bibliothèques réutilisables (utilitaires, services génériques, middlewares).
- **tests** : Répertoire dédié aux tests (unitaires, d'intégration, end-to-end).

Exemple de structure de dossiers

```
.
cmd
docs
generated
internal
    config
    db
    features
        admin
            api
            dto
            models
            repository
            services
            router.go
        auth
            api
            dto
            models
            repository
            services
            router.go
        user
            api
            dto
            models
            repository
            services
            router.go
    middleware
pkg
```

- cache
- cleanup
- e2e
- errors
- hash
- mailer
- ratelimiter
- rollback
- token
- transformer
- validators
- scripts
- tests
 - auth
 - categories
 - notifications
 - requests
 - services
- go.mod

Rôles des répertoires principaux

- **cmd** : Point d'entrée principal de l'application.
- **internal/config** : Chargement des variables d'environnement et configurations.
- **internal/db** : Initialisation et connexion à la base de données (PostgreSQL, SQLite).
- **internal/features** : Organisation par feature (admin, auth, user, etc.) avec :
 - **api** : Handlers et logique de routing.
 - **dto** : Data Transfer Objects pour validation et transport de données.
 - **models** : Entités persistantes (ex. mappage GORM).
 - **repository** : Accès aux données (DAO).
 - **services** : Logique métier reliant repository et API.
 - **router.go** : Fichier d'enregistrement des routes spécifiques.
- **internal/middleware** : Middlewares globaux (authentification, logs, rate limiting).

- **pkg** : Librairies et modules transversaux (ex. cache, mailer).
- **tests** : Tests classés par feature ou type.

2.2.1 Gestion de la Base de Données

Choix de SGBD

Pour la gestion des données, le choix s'est porté sur **PostgreSQL** pour les environnements de Test, de Développement et de Production. Ce SGBD offre plusieurs avantages :

- Gestion robuste des transactions respectant les propriétés ACID.
- Excellente gestion de la concurrence pour les environnements à fort trafic.
- Support d'extensions avancées telles que **PostGIS** pour des besoins de géolocalisation.

Migrations de Schéma

Le processus de gestion des schémas est automatisé à l'aide de l'outil **golang-migrate** (ou un équivalent comme Goose). Ce processus suit les étapes suivantes :

1. Création de scripts de migration versionnés sous la forme de fichiers **.up.sql** et **.down.sql**.
2. Utilisation d'une table dédiée (ex. **schema_migrations**) pour tracer les versions appliquées.
3. Intégration du processus dans la pipeline CI/CD afin de garantir la synchronisation automatique des schémas sur les environnements de production.

2.2.2 Endpoints REST et Organisation des Routes

Chaque fonctionnalité (feature) possède son propre fichier **router.go**, qui est ensuite enregistré dans le routeur principal, typiquement dans **cmd/main.go**.

Exemple d'enregistrement des routes

```
func RegisterAdminRoutes(r *gin.RouterGroup) {  
    admin := r.Group("/admin")
```

```

{
    admin.GET("/requests", GetAllRequests)
    admin.POST("/requests/:id/validate", ValidateRequest)
    admin.POST("/requests/:id/refuse", RefuseRequest)
    // ...
}
}

```

Standards REST

Les endpoints suivent les conventions REST, facilitant leur compréhension et leur utilisation :

- **GET /admin/requests** : Récupérer la liste de toutes les demandes.
- **POST /admin/requests/:id/validate** : Valider une demande spécifique.

Des fonctionnalités supplémentaires sont également intégrées :

- **Pagination**.
- **Filtrage** par statut, date de création, etc.

2.2.3 Authentification et Autorisation

Le système d'authentification repose sur l'utilisation de **JSON Web Tokens** (JWT), offrant un équilibre entre sécurité et simplicité.

Structure du Token

Chaque token contient des *claims* permettant de stocker les informations suivantes :

- **userId** : Identifiant unique de l'utilisateur.
- **role** : Rôle de l'utilisateur (**admin**, **user**, etc.).
- **exp** : Date d'expiration du token.

La signature HMAC-SHA256 est utilisée pour garantir l'intégrité du token.

Renouvellement de Token

Un mécanisme de **refresh tokens** est mis en place pour prolonger la durée de session de manière sécurisée sans exiger une ré-authentification complète.

Middlewares

Plusieurs middlewares sont utilisés pour sécuriser l'accès aux ressources :

- **AuthMiddleware** : Vérifie la validité et l'intégrité du token.
- **RoleMiddleware** : Restreint l'accès à certaines routes selon le rôle de l'utilisateur.
- **RateLimiterMiddleware** : Protège les endpoints sensibles contre les abus via un système de *rate limiting*, intégré depuis `pkg/ratelimiter`.

2.2.4 Couche de Services (Logique Métier)

Chaque fonctionnalité (**feature**) dispose de sa propre couche de services, encapsulant la logique métier spécifique à cette fonctionnalité.

Exemple : Service de Validation d'une Demande

```
package services

import (
    "context"
    "errors"

    "github.com/budgestin/internal/features/admin/models"
    "github.com/budgestin/internal/features/admin/repository"
)

type RequestService struct {
    repo repository.RequestRepository
}

func NewRequestService(r repository.RequestRepository)
↳ *RequestService {
    return &RequestService{repo: r}
}

func (s *RequestService) ValidateRequest(ctx context.Context,
↳ reqID uint, validatorID uint) error {
    request, err := s.repo.FindByID(ctx, reqID)
    if err != nil {
        return err
    }
}
```

```

    if request == nil {
        return errors.New("demande introuvable")
    }
    request.Status = "VALIDATED"
    return s.repo.Update(ctx, request)
}

```

Dans cet exemple, la logique métier (changement de statut) est centralisée dans le service, garantissant une séparation claire des responsabilités entre les couches API (contrôleurs), services (logique métier) et repository (accès aux données).

2.2.5 Couche de Repository (Accès aux Données)

La couche repository encapsule l'accès à la base de données via GORM.

Exemple : Repository de la demande

```

package repository

import (
    "context"
    "gorm.io/gorm"
    "estlin/internal/features/admin/models"
)

type RequestRepository interface {
    FindByID(ctx context.Context, id uint) (*models.Request,
        error)
    Update(ctx context.Context, req *models.Request) error
    // ...
}

type requestRepo struct {
    db *gorm.DB
}

func NewRequestRepo(db *gorm.DB) RequestRepository {
    return &requestRepo{db: db}
}

func (r *requestRepo) FindByID(ctx context.Context, id uint)
    (*models.Request, error) {

```

```

    var req models.Request
    if err := r.db.WithContext(ctx).First(&req, id).Error; err
    ↪ != nil {
        return nil, err
    }
    return &req, nil
}

func (r *requestRepo) Update(ctx context.Context, req
↪ *models.Request) error {
    return r.db.WithContext(ctx).Save(req).Error
}

```

2.2.6 DTO et Validation

Les DTO (Data Transfer Objects), situés sous `internal/features/<feature>/dto` définissent les schémas d'entrée et de sortie pour assurer une validation stricte.

Exemple de DTO

```

package dto

type RequestValidationDTO struct {
    RequestID uint    `json:"request_id" validate:"required"`
    Comment   string `json:"comment,omitempty"`
}

if err := validator.Struct(reqBody); err != nil {
    c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
    return
}

```

2.2.7 Gestion des Erreurs et Journalisation

Gestion des Erreurs

- Niveaux : Info, Warn, Error, Fatal.
- Standardisation : Réponses d'erreurs structurées en JSON.

```

{
    "error": "Le champ 'request_id' est requis."
}

```

Journalisation

- Middleware : Capture des requêtes HTTP, temps d'exécution et statuts.
- Format : Ajout de métadonnées (trace ID, user ID).

2.2.8 Sécurité et Bonnes Pratiques

- Validation stricte : Vérification rigoureuse des inputs côté serveur.
- Utilisation de GORM : Pour prévenir les injections SQL (requêtes paramétrées).
- Communication sécurisée : HTTPS/TLS pour chiffrer les échanges.
- Hachage des mots de passe : Utilisation de bcrypt avec un coût adapté.
- CORS : Configuration stricte pour limiter l'accès aux origines autorisées.
- Rate Limiting : Protection contre les attaques de type brute force et DDoS.
- Vérification des rôles et permissions : Mise en place de middlewares dédiés.

2.2.9 Tests et Qualité du Code

Tests Unitaires Ciblent des fonctions isolées (services, repository, etc.) via le package `testing` et des outils comme `testify`.

Tests d'Intégration Utilisation d'une base SQLite ou PostgreSQL en local pour tester les interactions globales (API + DB).

Tests End-to-End (E2E) Scénarios complets pour simuler les parcours utilisateurs. Possibilité d'utiliser des scripts (ex. `pkg/e2e`) ou des frameworks externes.

2.2.10 Gestion de la Configuration

Fichiers `.env` et Variables d'Environnement

- Développement : Utilisation de fichiers `.env` avec `godotenv`.
- Production : Variables d'environnement gérées via Docker, orchestrateurs, ou gestionnaires de secrets.

Exemple

```
ACCESS_JWT_SECRET=  
REFRESH_JWT_SECRET=  
APP_PASSWORD=  
EMAIL_SENDER=  
INTERNAL_TOKEN=  
PRODUCTION=  
DATABASE_HOST=  
DATABASE_USER=  
DATABASE_PASSWORD=  
DATABASE_NAME=  
DATABASE_PORT=  
DATABASE_SSL_MODE=
```

2.2.11 Problèmes Rencontrés et Solutions Apportées

Gestion concurrente de goroutines

- **Problème** : Fuites de goroutines lors de traitements lourds.
- **Solution** : Utilisation de contextes de cancellation et synchronisation via mutex/channels.

Performances de la base

- **Problème** : Temps de réponse élevé sur certaines requêtes.
- **Solution** : Ajout d'index, optimisation des requêtes et implémentation d'un cache (**pkg/cache**).

Gestion des rôles multiples

- **Problème** : Duplication de la logique d'autorisation.
- **Solution** : Middleware générique centralisant la vérification des rôles.

Migrations complexes

- **Problème** : Coordination difficile entre plusieurs environnements.
- **Solution** : Automatisation des migrations via CI/CD, avec scripts de rollback et outils de migration.

2.2.12 Conclusion

L'architecture backend de BudgESTIN est conçue pour être modulaire, scalable et sécurisée. Grâce à l'utilisation de Go, le système bénéficie de performances élevées et d'une gestion efficace de la concurrence. La séparation en features (**admin**, **auth**, **user**) et l'intégration de bonnes pratiques de validation, testing et déploiement garantissent une application robuste et maintenable.

Conclusion Générale

Ce stage nous a offert l'opportunité de mettre en application nos connaissances théoriques et techniques acquises tout au long de notre formation. À travers la conception et la mise en œuvre de l'application **BudgESTIN**, nous avons pu aborder les différentes étapes d'un projet logiciel complet, depuis la phase de conception jusqu'au déploiement, tout en respectant les bonnes pratiques de développement backend.

Bilan du Projet

Le projet BudgESTIN a permis de concevoir une architecture backend modulaire, robuste et évolutive, s'appuyant sur les atouts du langage **Go**, particulièrement adapté aux systèmes nécessitant une gestion efficace de la concurrence et de hautes performances.

Nous avons mis en place une architecture orientée **features**, favorisant la séparation des responsabilités et la maintenabilité du code. Cette structuration, associée à l'utilisation de GORM pour la couche d'accès aux données, nous a permis de répondre aux besoins fonctionnels tout en garantissant la sécurité et la performance de la plateforme.

Les principaux axes techniques abordés dans ce projet sont :

- **Modularité et scalabilité** : Chaque fonctionnalité est isolée dans un module spécifique, facilitant l'évolution future du projet.
- **Performance et gestion de la concurrence** : Utilisation de **goroutines** pour le traitement parallèle des tâches et optimisation de la gestion des ressources via des **contextes**.
- **Sécurisation des flux** : Application des bonnes pratiques de sécurité, incluant la validation stricte des entrées, la gestion sécurisée des sessions et la protection contre les attaques courantes.
- **Fiabilité et qualité logicielle** : Mise en place de tests unitaires et de tests d'intégration pour chaque composant clé de la plateforme.

- **Portabilité et déploiement** : Utilisation de Docker pour assurer la portabilité entre les environnements de développement, de test et de production.

Compétences Acquises

Ce projet nous a permis d'acquérir et de consolider un ensemble de compétences techniques et méthodologiques essentielles pour la gestion d'un projet logiciel en équipe, notamment :

- **Conception et structuration d'un backend complet** : Adoption d'une architecture hexagonale adaptée aux besoins métiers.
- **Gestion des flux et sécurisation des données** : Mise en place de couches de validation, de gestion des erreurs et de contrôles d'accès robustes.
- **Travail collaboratif** : Utilisation de Git pour la gestion de version et la coordination, avec une organisation en branches par fonctionnalités et la revue de code systématique.
- **Intégration et déploiement** : Gestion de la configuration par environnement, utilisation de Docker Compose pour la coordination des services et anticipation des contraintes de production.

Difficultés Rencontrées et Solutions Apportées

La réalisation de BudgESTIN a été confrontée à plusieurs défis techniques et organisationnels :

- **Concurrence et fuites de goroutines** : Certaines tâches longues ont occasionné des fuites de ressources, nécessitant la mise en place de **contextes** avec annulation et d'une synchronisation fine via **channels** et **WaitGroup**.
- **Optimisation des performances** : Certaines requêtes critiques ont montré des latences élevées, ce qui nous a conduits à optimiser les requêtes SQL, ajouter des index et introduire un système de cache local.
- **Sécurisation globale** : La gestion des authentifications, des tokens JWT et la sécurisation des communications (HTTPS) ont nécessité une attention particulière pour garantir la confidentialité et l'intégrité des données.

- **Coordination en équipe** : La répartition des tâches et la gestion des conflits de fusion ont exigé une discipline rigoureuse dans la gestion de version et l'organisation des sprints.

Ces obstacles ont été surmontés grâce à une collaboration étroite, une communication continue entre les membres de l'équipe et une recherche constante de solutions adaptées.

Perspectives et Évolutions Futures

Même si une première version fonctionnelle de BudgESTIN a été livrée, plusieurs pistes d'amélioration et d'évolution restent envisageables pour pérenniser et enrichir la plateforme :

- **Automatisation des déploiements** : Mise en place d'une chaîne CI/CD intégrant des tests automatisés et des déploiements continus.
- **Observabilité et monitoring** : Intégration de solutions de monitoring centralisé (Prometheus, Grafana) pour un suivi en temps réel des performances et des incidents.
- **Optimisation de la scalabilité** : Étude de l'adoption d'une architecture microservices pour certaines fonctionnalités critiques afin d'améliorer l'évolutivité.
- **Extension fonctionnelle** : Ajout de nouveaux modules, notamment pour la génération automatisée de rapports analytiques et la prévision budgétaire à partir des historiques de données.

Conclusion

En conclusion, ce projet nous a permis de mener à bien un développement backend complet, alliant rigueur méthodologique, maîtrise technique et respect des bonnes pratiques de développement logiciel. Cette expérience nous a permis de nous confronter aux réalités d'un projet d'envergure, tout en consolidant nos compétences dans la gestion de projet en équipe, la résolution de problèmes techniques et la mise en œuvre de solutions robustes et évolutives.

Nous remercions l'ensemble de l'équipe encadrante pour leur accompagnement, leurs conseils et leur disponibilité tout au long de ce projet. Leur expertise et leurs retours constructifs ont largement contribué à la réussite de cette mission.